

SQL – Procedimientos y Funciones



Objetivo: Crear y gestionar procedimientos almacenados y funciones para ejecutar operaciones complejas de manera eficiente, y entender el uso de SQL dinámico para generar consultas en tiempo de ejecución.



SQL a fondo... Dominando consultas avanzadas y técnicas dinámicas

IDENTIFICADORES

*Los **identificadores** son nombres utilizados para identificar objetos dentro de una base de datos, como tablas, columnas, procedimientos almacenados, funciones, índices, vistas, entre otros. Los identificadores permiten referirse a estos objetos de manera única dentro de un contexto.*

Reglas para los identificadores en SQL Server:

1. **Longitud máxima:** 128 caracteres.
2. **Caracteres permitidos:**
 - Pueden comenzar con:
 - una letra (A-Z, a-z),
 - un signo de arroba (@),
 - un subrayado (_),
 - un símbolo de número (#),
 - Caracteres posteriores pueden incluir números (0-9).
3. **Sin espacios ni caracteres especiales:** Los nombres de identificadores no pueden incluir espacios en blanco ni caracteres especiales como %, \$, &, etc.
4. **Palabras reservadas:** No se pueden usar palabras clave o reservadas del sistema (como SELECT, TABLE, WHERE)



a menos que se encierran entre corchetes o comillas dobles.

VARIABLES

Las variables son objetos que se utilizan para almacenar temporalmente valores de datos mientras se ejecuta un script o procedimiento.

Son muy útiles para realizar cálculos intermedios, almacenar resultados de consultas o manejar datos dentro de bloques de control como IF, WHILE, y otros.

Reglas para Variables en SQL Server:

1. **Longitud máxima:** 128 caracteres.
2. **Caracteres permitidos:**
 - Siempre debe comenzar con @
 - El segundo carácter puede ser:
 - una letra (A-Z, a-z),
 - un subrayado (_),
 - Caracteres posteriores pueden incluir números (0-9).
3. **Sin espacios ni caracteres especiales:** Los nombres de identificadores no pueden incluir espacios en blanco ni caracteres especiales como %, \$, &, etc.
4. **Palabras reservadas:** No se pueden usar palabras clave o reservadas del sistema (como SELECT, TABLE, WHERE)

```
DECLARE @PrecioMin DECIMAL(10,2);  
SET @PrecioMin = 50.00;
```

```
SELECT Nombre, Precio  
FROM Productos  
WHERE Precio > @PrecioMin;
```

Situación	SET	SELECT
Asignar un valor directo a una variable	Correcto	Correcto
Asignar una variable por vez	Sí, es lo habitual	También puede hacerlo
Asignar varias variables en una misma sentencia	No	Sí
La consulta devuelve una fila y una columna	Asigna correctamente	Asigna correctamente
La consulta devuelve una fila y varias columnas	Error, salvo que cada SET tome una sola columna por separado	Correcto si cada columna se asigna a una variable distinta
La consulta no devuelve filas	La variable queda en NULL	La variable conserva el valor anterior
La consulta devuelve varias filas y una columna	Error	Asigna varias veces y queda el último valor procesado
La consulta devuelve varias filas y varias columnas	Error	Asigna varias veces cada variable y queda el último conjunto de valores procesado
La subconsulta devuelve más de un valor para una sola variable	Error	No aplica igual; si se usa como asignación directa puede ir sobrescribiendo
Control y previsibilidad	Más estricto y seguro para un único valor	Más flexible, pero puede ser riesgoso si devuelve varias filas
Uso recomendado	Cuando se espera un único valor escalar	Cuando se quieren asignar varias variables desde una misma fila
Riesgo principal	Error si la consulta devuelve más de un valor	Puede dejar valores anteriores si no hay filas o tomar un “último” valor no garantizado si hay varias filas

PARRAFOS

En T-SQL (Transact-SQL), los párrafos se refieren a bloques de instrucciones o sentencias SQL que se agrupan para ser ejecutados juntos. Pueden ser definidos de forma implícita o explícita.

Párrafo implícito:

Un párrafo implícito es un conjunto de instrucciones que no está explícitamente delimitado por palabras clave como BEGIN y END, pero que se agrupan automáticamente dentro de la interfaz de consulta o el entorno de ejecución. Solo incluye 1 instrucción.

Párrafo explícito:

Un párrafo explícito es un bloque de instrucciones que está delimitado claramente usando las palabras clave BEGIN y END.



```
DECLARE @Nombre NVARCHAR(50);  
SET @Nombre = 'María';  
PRINT @Nombre;
```

```
BEGIN  
    DECLARE @Nombre NVARCHAR(50);  
    SET @Nombre = 'María';  
    PRINT @Nombre;  
END;
```

ESTRUCTURAS DE DECISION

IF..ELSE: Permite ejecutar un bloque de código cuando una condición es verdadera, y opcionalmente, otro bloque de código cuando la condición es falsa.

Las estructuras de decisión son construcciones que permiten controlar el flujo del programa basándose en condiciones lógicas. Estas estructuras te permiten ejecutar diferentes bloques de código dependiendo de si una condición es verdadera o falsa.

```
IF condición_booleana
    BEGIN    -- Bloque de código a ejecutar si la
              condición es verdadera
    END
ELSE
    BEGIN    -- Bloque de código a ejecutar si la
              condición es falsa
    END
```

```
DECLARE @Edad INT = 20;
IF @Edad >= 18
    PRINT 'Es mayor de edad.';
ELSE
    PRINT 'Es menor de edad.';
```

- Si el párrafo contiene una sola instrucción, se pueden omitir el BEGIN y el END
- Cuando la expresión booleana es verdadera debe tener código sin excepción
- Tener en cuenta que los nulos pueden alterar la lógica de dos estados

ESTRUCTURAS DE DECISION

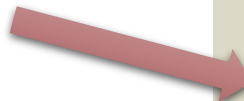
CASE: evalúa una expresión y devuelve diferentes resultados dependiendo del valor de la expresión o condición.

- **Sencilla:** Compara una expresión con varios valores y devuelve el resultado correspondiente.
- **Búsqueda:** Evalúa una serie de condiciones y devuelve el resultado cuando una condición es verdadera

Las estructuras de decisión son construcciones que permiten controlar el flujo del programa basándose en condiciones lógicas. Estas estructuras te permiten ejecutar diferentes bloques de código dependiendo de si una condición es verdadera o falsa.



```
DECLARE @Calificacion INT = 85;
SELECT
    CASE
        WHEN @Calificacion >= 90 THEN 'A'
        WHEN @Calificacion >= 80 THEN 'B'
        WHEN @Calificacion >= 70 THEN 'C'
        ELSE 'F'
    END AS CalificacionLetra;
```



```
DECLARE @DiaSemana INT = 3;
SELECT
    CASE @DiaSemana
        WHEN 1 THEN 'Lunes'
        WHEN 2 THEN 'Martes'
        WHEN 3 THEN 'Miércoles'
        WHEN 4 THEN 'Jueves'
        WHEN 5 THEN 'Viernes'
        ELSE 'Fin de semana'
    END AS Dia;
```


ESTRUCTURAS DE DECISION

WHILE: El bucle WHILE es una estructura de control de repetición que ejecuta un bloque de código repetidamente mientras una condición sea verdadera.

```
WHILE condición_booleana {  
    { sentencia | bloque }  
    [ BREAK ]  
    { sentencia | bloque }  
    [ CONTINUE ]  
    { sentencia | bloque } }
```

- **BREAK:** sale del bucle inmediatamente y se ejecutan las sentencias que figuran después del final del bloque del WHILE.
- **CONTINUE** reinicia el bloque WHILE omitiendo las instrucciones posteriores al continue.

Las estructuras de decisión son construcciones que permiten controlar el flujo del programa basándose en condiciones lógicas. Estas estructuras te permiten ejecutar diferentes bloques de código dependiendo de si una condición es verdadera o falsa.

```
DECLARE @Contador INT = 1;  
WHILE @Contador <= 10  
BEGIN  
    IF @Contador = 3  
        BEGIN  
            SET @Contador = @Contador + 1;  
            CONTINUE; -- Salta es igual a 3  
        END  
    IF @Contador = 5  
        BEGIN  
            BREAK; -- Sale del bucle es 5  
        END  
    PRINT 'Iteración número ' +  
        CAST(@Contador AS NVARCHAR(10));  
    SET @Contador = @Contador + 1;  
END
```


ESTRUCTURAS DE DECISION

TRY...CATCH: Es una estructura de control que permite capturar y manejar errores que puedan ocurrir durante la ejecución de un bloque de código..

```
BEGIN TRY
    -- Bloque de código que puede
    -- generar un error
END TRY
BEGIN CATCH
    -- Bloque de código que se ejecuta si
    -- ocurre un error
END CATCH
```

Las estructuras de decisión son construcciones que permiten controlar el flujo del programa basándose en condiciones lógicas. Estas estructuras te permiten ejecutar diferentes bloques de código dependiendo de si una condición es verdadera o falsa.

```
DECLARE @Resultado INT;
BEGIN TRY
    -- Intentar dividir por cero, lo que
    -- generará un error
    SET @Resultado = 10 / 0;
END TRY
BEGIN CATCH
    -- Capturar y manejar el error
    PRINT 'Ocurrió un error: ' +
        ERROR_MESSAGE();
    SET @Resultado = 0
    PRINT 'Valor de @Resultado: ' +
        CAST(@Resultado AS NVARCHAR(10));
END CATCH
```



VARIABLES DEL SISTEMA

En SQL Server, existen varias variables de sistema que proporcionan información útil sobre el estado del servidor, el contexto de la sesión, las transacciones, y más. Estas variables son predefinidas por el sistema y comienzan con @@.

Variable	Descripción
@@ERROR	Código de error de la última instrucción ejecutada.
@@ROWCOUNT	Número de filas afectadas por la última instrucción.
@@IDENTITY	Último valor de identidad generado en una tabla.
@@TRANCOUNT	Número de transacciones activas en la sesión.
@@VERSION	Información sobre la versión de SQL Server.
@@SPID	ID del proceso de la sesión actual.
@@TIMETICKS	Número de "ticks" del reloj del servidor.
@@CPU_BUSY	Tiempo de CPU utilizado desde que se inició SQL Server.
@@MAX_CONNECTIONS	Número máximo de conexiones permitidas en el servidor.
@@LANGUAGE	Idioma actual de la sesión.
@@SERVERNAME	Nombre del servidor SQL Server actual.
@@TOTAL_READ	Número total de lecturas de disco realizadas desde que se inició el servidor.
@@TOTAL_WRITE	Número total de escrituras en disco realizadas desde que se inició el servidor.

```
BEGIN TRY
UPDATE Productos SET Precio = Precio / 0
WHERE ID_Producto = 1;
END TRY
BEGIN CATCH
PRINT 'Código de error: ' +
CAST(@@ERROR AS NVARCHAR(10));
END CATCH;
SELECT message_id, severity, text
FROM sys.messages
WHERE message_id = 8134 and language_id
in (1033, 3082);
```

PROCEDIMIENTOS ALMACENADOS

Un procedimiento almacenado en SQL Server es un conjunto de sentencias SQL predefinidas que se almacenan y se ejecutan en el servidor de bases de datos. Los procedimientos almacenados permiten realizar operaciones complejas y reutilizables, como consultas, actualizaciones, inserciones, eliminaciones y lógica de control de flujo, todo en una única unidad de ejecución.

Características:

- **Reutilizable**
- **Almacenado en el servidor**
- **Parámetros**
- **Mejora del rendimiento:** SQL Server compila y optimiza los procedimientos almacenados
- **Seguridad:** SQL controla el acceso a los datos, ya que se puede otorgar permisos para ejecutar el procedimiento sin que el usuario tenga acceso directo a las tablas subyacentes.
- **Cantidad máxima de caracteres:** 2 GB (gigabytes)
- **Máximo de parámetros:** 2100 parámetros.

```
CREATE PROCEDURE sp_ObtenerCliente
    @ID_Cliente INT
AS
BEGIN
    SELECT NombreCliente, Ciudad
    FROM Clientes
    WHERE ID_Cliente = @ID_Cliente;
END;
```

PROCEDIMIENTOS ALMACENADOS

Estructura:

```
CREATE PROCEDURE NombreProcedimiento
    [ { @parámetro tipo_de_dato }
        [ = default ] – Valor por defecto
        [ OUTPUT ] – Parámetro de salida
    [ , ... n ]
    [ WITH ENCRYPTION ]
    [ FOR REPLICATION ]
    AS
    BEGIN
        -- Bloque de sentencias SQL + T-SQL
        [ RETURN [entero] ]
    END;
```

DROP: Elimina procedimiento

ALTER: Modifica procedimiento

(requiere la declaración completa)

Ejecución:

```
EXEC NombreProcedimiento
    [ @parámetro = { Valor }
    [, ... n ] ];
```

**** Tener en cuenta el orden cuando se invoca el nombre del parámetro explícitamente**

FUNCIONES DEL SISTEMA

Funciones SQL



Las funciones SQL definidas por el sistema se conocen como funciones del sistema. En otras palabras, todas las funciones integradas admitidas por el servidor se denominan **funciones del sistema**.

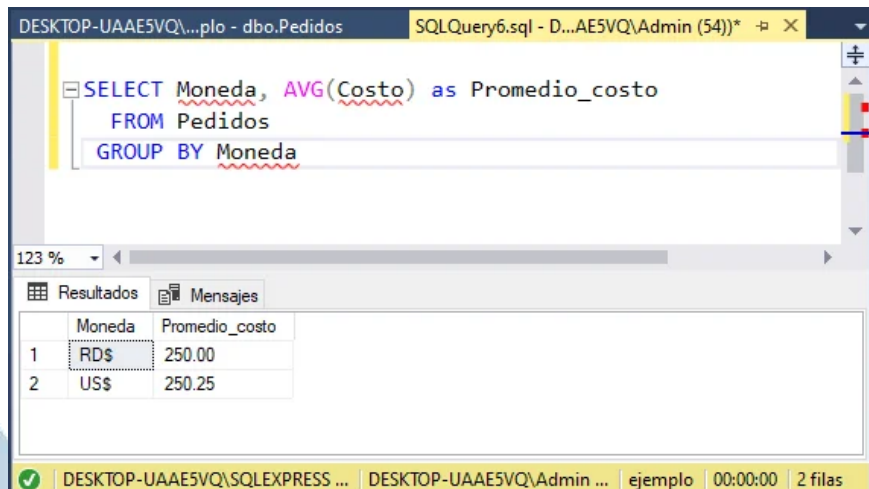
Las funciones integradas nos ahorran tiempo mientras realizamos la tarea específica. Estos tipos de funciones SQL, generalmente funcionan con la instrucción SQL **SELECT** para calcular valores y manipular datos.

- Funciones de agregado
- Funciones de cadena
- Funciones de fecha y hora
- Funciones de convención
- Funciones matemáticas
- Funciones de categorías
- Funciones de analíticas
- Funciones de configuración

FUNCIONES DEL SISTEMA

- Funciones de agregado
- Funciones de cadena
- Funciones de fecha y hora
- Funciones de convención
- Funciones matemáticas
- Funciones de categorías
- Funciones de analíticas
- Funciones de configuración

- AVG()– devuelve el promedio de un conjunto.
- COUNT()– devuelve el número de elementos en un conjunto.
- MAX()– devuelve el valor más alto (máximo) en un conjunto de valores.
- MIN()– devuelve el valor mínimo en un conjunto
- SUM()– devuelve la suma de todos o valores distintos en un conjunto



The screenshot shows a SQL Server query window with the following query:

```
SELECT Moneda, AVG(Costo) as Promedio_costo
FROM Pedidos
GROUP BY Moneda
```

The results are displayed in a table with two columns: Moneda and Promedio_costo.

Moneda	Promedio_costo
RD\$	250.00
US\$	250.25

La función suma en SQL SERVER

id	Nombre	Genero	Año	Sueldo
1	Jolly	F	20	5200
2	José	M	22	6450
3	Rosa	F	25	6100
4	Laura	F	18	4030
5	Alan	M	20	5000
6	Katy	F	22	6000
7	Joseph	M	18	6430
8	Antonio	M	23	5013
9	ANA	M	21	6090

```
SELECT sum(Sueldo) as
RESULTADO
FROM EMPLEADOS
```

RESULTADO
50313

FUNCIONES DEL SISTEMA

- Funciones de agregado
- Funciones de cadena
- Funciones de fecha y hora
- Funciones de convención
- Funciones matemáticas
- Funciones de categorías
- Funciones de analíticas
- Funciones de configuración

```
SELECT TOP 10 LEN([Nombre])  
AS Longitud Nombre  
FROM Empleado  
ORDER BY LEN([Nombre]) DESC
```

- CHARINDEX() – Retorna la posición inicial.
- CONCAT() – Devuelve una cadena como resultado de una concatenación.
- LEFT() – Extraiga un número caracteres comenzando desde la izquierda.
- LEN() – Devuelve el número de caracteres, sin incluir los espacios finales.
- LOWER() – Devuelve una expresión de caracteres en minúsculas.
- LTRIM() – Eliminar los espacios iniciales de una cadena de caracteres.
- SUBSTRING() – devuelve parte de un carácter.
- REPLACE() – Reemplaza los valores de una cadena especificada.
- RIGHT() – Extrae caracteres del lado derecho de una cadena de texto.
- RTRIM() – Para truncar los espacios en blanco finales de una cadena.
- UPPER() – Convertir una cadena a mayúsculas

FUNCIONES DEL SISTEMA

- Funciones de agregado
- Funciones de cadena
- Funciones de fecha y hora
- Funciones de convención
- Funciones matemáticas
- Funciones de categorías
- Funciones de analíticas
- Funciones de configuración

```
SELECT DATEADD(month, 11, '2019/01/05');
```

Resultado: 2019-12-05 00:00:00.000

- DATEADD() – Modifica una fecha agregando un valor.
- DATEDIFF() –Calcula la diferencia entre dos fechas.
- DATEFROMPARTS() – Devuelve un valor de fecha para el año, mes o día.
- DATENAME() – Devuelve un nombre que representa la parte de fecha.
- DAY() – Devuelve un entero que representa el día de una fecha dada.
- EOMONTH() – Devuelve el el último día del mes.
- GETDATE() – Retorna la fecha y hora actual en el servidor SQL.
- ISDATE() – Verifica si un valor es una FECHA, HORA o FECHA HORA válida
- MONTH() – Devuelve un número entero que representa el mes de una fecha.
- SYSDATETIME() – Devuelve un valor datetime2 que contiene la fecha y la hora.
- YEAR() – Devuelve un número entero que representa el año de una fecha dada.

FUNCIONES DEL SISTEMA

- Funciones de agregado
- Funciones de cadena
- Funciones de fecha y hora
- Funciones de convención
- Funciones matemáticas
- Funciones de categorías
- Funciones de analíticas
- Funciones de configuración

`SELECT CAST( expresión AS tipo_de_datos );`

- CAST() – convierte una expresión de un tipo de datos a otro.
- CONVERT() – Similar a CAST, pero con estilo de formato para fecha y la hora.
- PARSE() – Para convertir una cadena en tipos de fecha/hora y números.
- TRY_CAST() – Igual a cast, convierte una expresión de un tipo de datos a otro.
- TRY_CONVERT() – Similar a CONVERT, pero devuelve NULL y no error.
- TRY_PARSE() – Similar a PARSE, pero devuelve NULL y no error

```
SELECT CAST(25.5 AS FLOAT) AS NUM_FLOTANTE,  
       CAST('2023-01-01' AS DATE) AS FECHA,  
       CAST('2023-01-01' AS datetime) AS FECHA_HORA,  
       CAST(1454 AS CHAR(10)) AS FORMATO_TEXTO,  
       CAST(13 AS DECIMAL(4,2)) AS DECIMALES,  
       CAST(13.05 AS INT) AS ENTERO
```

FUNCIONES DEL SISTEMA

- Funciones de agregado
- Funciones de cadena
- Funciones de fecha y hora
- Funciones de convención
- Funciones matemáticas
- Funciones de categorías
- Funciones de analíticas
- Funciones de configuración

`SELECT ROUND(123.4567, 2) AS Resultado;`

- ABS() – obtiene el valor absoluto o el valor absoluto positivo.
- ASIN() – Obtiene el ángulo en radianes, se conoce como arcoseno.
- EXP() – obtiene un valor exponencial de la expresión flotante dada.
- LOG() – obtiene el logaritmo natural de la expresión flotante dada.
- ROUND() – Redondea a la longitud o precisión especificada.
- SIGN() – Obtiene el signo de la expresión, Positivo (+), Negativo (-) o Cero (0).
- SIN() – Devuelve el valor trigonométrico SINE en radianes para el ángulo dado.
- SQRT() – Proporciona la raíz cuadrada para un número o valor dado.
- SQUARE() – calcula el cuadrado de un valor específico o un número individual.
- TAN() – Obtiene el valor en radianes de la tangente trigonométrica del ángulo dado.

FUNCIONES DEL SISTEMA

- Funciones de agregado
- Funciones de cadena
- Funciones de fecha y hora
- Funciones de convención
- Funciones matemáticas
- Funciones de categorías
- Funciones de analíticas
- Funciones de configuración

- ROW_NUMBER () – Serializa las filas del conjunto de resultados particionado.
- RANK () – Devuelve el rango de un valor en una lista determinada.
- DENSE_RANK () – Devuelve la posición relativa de las filas dentro del conjunto.
- NTILE () – divide las filas en grupos.

```
SELECT
  ID_Empleado,
  NombreEmpleado,
  Sueldo,
  ROW_NUMBER() OVER (ORDER BY Sueldo
DESC) AS NumeroFila,
  RANK() OVER (ORDER BY Sueldo DESC)
  AS Rango,
  DENSE_RANK() OVER (ORDER BY Sueldo
DESC) AS RangoDenso,
  NTILE(3) OVER (ORDER BY Sueldo DESC)
  AS Tercil
FROM Empleados;
```

ID_Empleado	NombreEmpleado	Sueldo	NumeroFila	Rango	RangoDenso	Tercil
1	Juan	7000	1	1	1	1
2	Ana	6000	2	2	2	1
3	María	6000	3	2	2	1
4	Pedro	5500	4	4	3	2
5	Carlos	5500	5	4	3	2
6	Sofía	5000	6	6	4	3
7	Luis	4500	7	7	5	3

FUNCIONES DEL SISTEMA

- Funciones de agregado
- Funciones de cadena
- Funciones de fecha y hora
- Funciones de convención
- Funciones matemáticas
- Funciones de categorías
- Funciones de analíticas
- Funciones de configuración

- CUME_DIST() –calcula valores de distribución acumulativos de filas.
- FIRST_VALUE() – Para devolver el primer valor en un conjunto ordenado
- LAST_VALUE() –Para devolver el ultimo valor en un conjunto ordenado
- LAG() – Accede a los datos de la fila anterior.
- LEAD() – Accede a los datos de la fila siguiente.
- PERCENT_RANK() – Similar a la función CUME_DIST, que esta al principio.
- PERCENTILE_CONT() – calcula un percentil en una distribución de la columna.
- PERCENTILE_DISC() – calcula un percentil específico para valores ordenados en un conjunto de filas.

FUNCIONES DEL SISTEMA

SELECT

```
ID_Empleado,  
NombreEmpleado,  
Sueldo,  
CUME_DIST() OVER (ORDER BY Sueldo DESC) AS DistribucionAcumulada,  
FIRST_VALUE(Sueldo) OVER (ORDER BY Sueldo DESC) AS PrimerSueldo,  
LAST_VALUE(Sueldo) OVER (ORDER BY Sueldo ASC) AS UltimoSueldo,  
LAG(Sueldo, 1) OVER (ORDER BY Sueldo DESC) AS SueldoAnterior,  
LEAD(Sueldo, 1) OVER (ORDER BY Sueldo DESC) AS SueldoSiguiente,  
PERCENT_RANK() OVER (ORDER BY Sueldo DESC) AS RangoPorcentual,  
PERCENTILE_CONT(0.5) WITHIN GROUP (ORDER BY Sueldo) OVER () AS Mediana  
FROM Empleados;
```

ID_Empleado	NombreEmpleado	Sueldo	DistribucionAcumulada	PrimerSueldo	UltimoSueldo	SueldoAnterior	SueldoSiguiente	RangoPorcentual	Mediana
1	Juan	7000	1.0000	7000	4500	NULL	6000	0.0000	5500
2	Ana	6000	0.7143	7000	4500	7000	6000	0.1667	5500
3	María	6000	0.7143	7000	4500	6000	5500	0.1667	5500
4	Pedro	5500	0.4286	7000	4500	6000	5500	0.5000	5500
5	Carlos	5500	0.4286	7000	4500	5500	5000	0.5000	5500
6	Sofia	5000	0.1429	7000	4500	5500	4500	0.8333	5500
7	Luis	4500	0.0000	7000	4500	5000	NULL	1.0000	5500

FUNCIONES DEL SISTEMA

- Funciones de agregado
- Funciones de cadena
- Funciones de fecha y hora
- Funciones de convención
- Funciones matemáticas
- Funciones de categorías
- Funciones de analíticas
- Funciones de configuración

- @@VERSION() – Devuelve información sobre la versión de SQL Server.
- @@SPID() – Devuelve el identificador de proceso (SPID) del proceso actual.
- @@SERVERNAME() – esta función devuelve el nombre del servidor de SQL.
- @@TOTAL_MEMORY() – Devuelve la cantidad total de memoria en megabytes.
- @@CPU_BUSY() – Devuelve el tiempo total de CPU utilizado en milisegundos.
- @@MAX_CONNECTIONS() – Devuelve el número máximo de conexiones simultáneas permitidas en el servidor.

FUNCIONES

Las funciones están diseñadas para devolver un valor o un conjunto de resultados. Las funciones pueden ser de dos tipos principales:

- **Funciones escalares:** Devuelven un solo valor.
- **Funciones con valores de tabla:** Devuelven un conjunto de resultados.
 - *Inline table-valued function (iTVF)*
 - *Multistatement table-valued function (mTVF) - multideclarativa*

Característica	Procedimiento almacenado	Función
Devuelve	Múltiples resultados, valores o nada (usando RETURN)	Un solo valor o un conjunto de resultados (una tabla)
Usos	Manipulación de datos (INSERT, UPDATE, DELETE), transacciones	Consultas y cálculos. No se permite INSERT, UPDATE, o DELETE
Acepta parámetros	Sí, con valores por defecto	Sí, pero no admite parámetros OUTPUT
Se usa en	Llamadas directas con EXEC	Se puede usar en SELECT, WHERE, JOIN, etc.
Tipos de datos permitidos	Sin restricciones	Devuelve un tipo de datos específico o una tabla

FUNCIONES

Estructura escalares:

```
CREATE FUNCTION NombreFuncion  
    [ { @parámetro tipo_de_dato }  
        [ = default ] – Valor por defecto  
        [ , ... n ]  
RETURNS tipo_de_dato  
[ AS ]  
BEGIN  
    RETURN expresión_escalar  
END;
```

```
SELECT NombreFuncion (valor_parametro)
```

FUNCIONES

Estructura con valores de tabla en línea (Inline table-valued function (iTVF)):

```
CREATE FUNCTION NombreFuncion  
    [ { @parámetro tipo_de_dato }  
        [ = default ] – Valor por defecto  
        [ , ... n ]  
RETURNS TABLE  
AS  
RETURN  
BEGIN  
    SELECT columnas  
    FROM tabla  
    WHERE condición;  
END;
```

```
SELECT *  
FROM NombreFuncion (valor_parametro)
```

FUNCIONES

Estructura con valores de tabla multideclarativa (Multistatement table-valued function (mTVF)):

```
CREATE FUNCTION NombreFuncion
```

```
    [ { @parámetro tipo_de_dato } [ = default ] [ , ... n ]
```

```
    RETURNS @TablaResultado TABLE
```

```
        ( columna1 tipo_de_dato,  
          columna_n tipo_de_dato,  
        )
```

```
AS
```

```
BEGIN
```

```
    INSERT INTO @TablaResultado
```

```
        SELECT columnas
```

```
    FROM tabla
```

```
    WHERE condición;
```

```
    RETURN;
```

```
END;
```

```
SELECT *
```

```
FROM NombreFuncion (valor_parametro)
```

SQL - Structured Query Language



Bibliografía:

Introducción a los Sistemas de Bases de Datos - C. J. Date – 7º Edición

- **CAPÍTULO 8: Integridad**
- **CAPÍTULO 13: Modelado semántico**
- **CAPÍTULO 20: Bases de datos distribuidas**

Sistemas de Bases de Datos – Conceptos Fundamentales - Elmasri / Navathe – 5º Edición.

- **Capítulo 9: Introducción a las técnicas de programación SQL.**
- **Capítulo 20: Modelos de datos mejorados para aplicaciones avanzadas**

Microsoft Docs – SQL Server: SQL Dynamic

Próximo encuentro



Clase 10: SQL y Seguridad

Cursores, transacciones, triggers, esquemas y sinónimos.

Grant y Revoke (permisos).

Ejercitación práctica.